CLASS ACTIVITY -15.07.2026

Name : S.Padmaja

Reg no:192424159

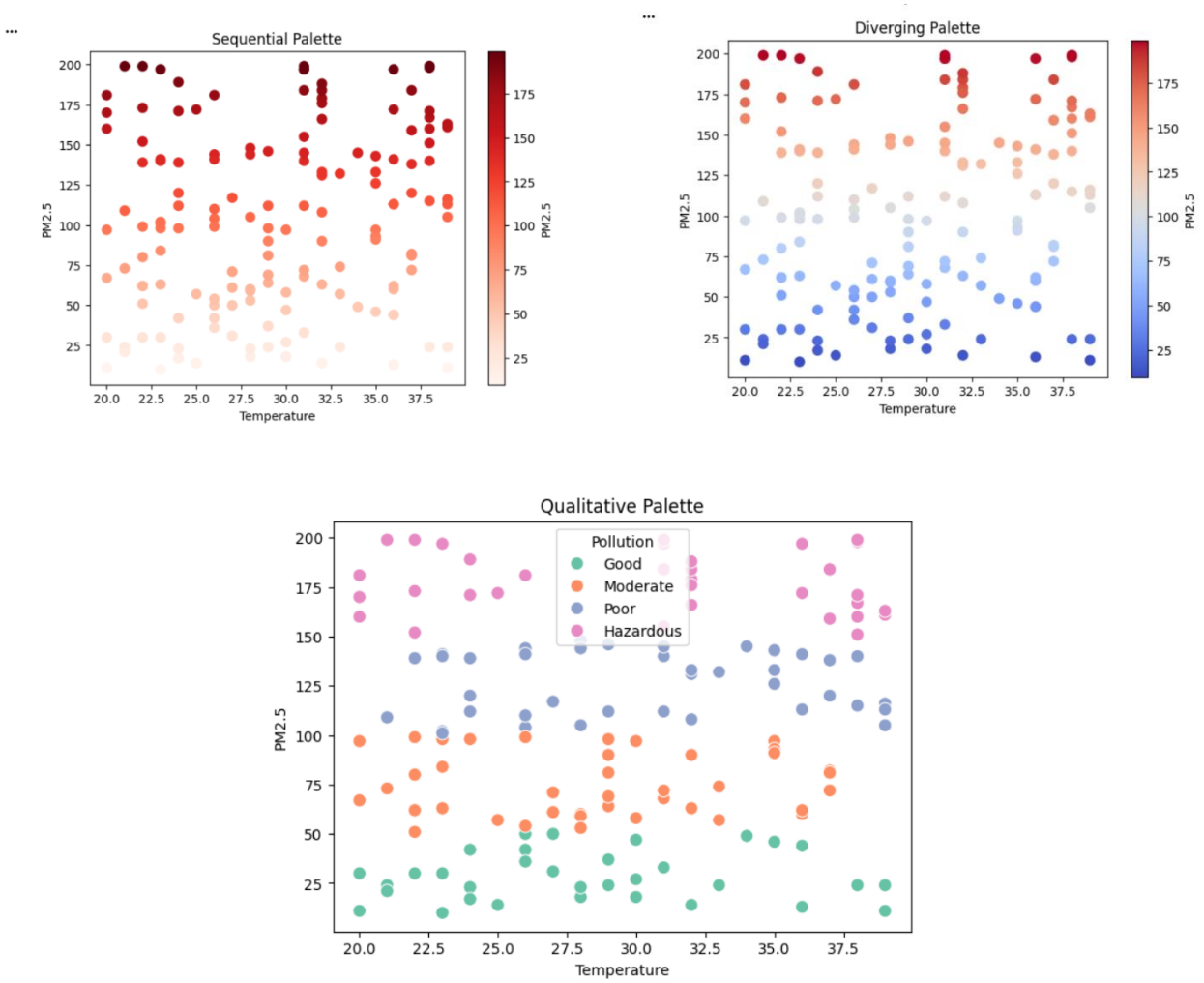
SCENARIO 1: SMART CITY AIR QUALITY MONITORING

CODE : import pandas as pd

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
np.random.seed(42)
df = pd.DataFrame({
   'PM2.5': np.random.randint(10, 200, 150),
   'NO2': np.random.randint(5, 120, 150),
   'CO': np.random.uniform(0.5, 5, 150),
   'Temperature': np.random.randint(20, 40, 150),
   'Humidity': np.random.randint(30, 90, 150)
})
df['Pollution'] = pd.cut(df['PM2.5'],
                         bins=[0, 50, 100, 150, 250],
                         labels=['Good', 'Moderate', 'Poor', 'Hazardous'])
plt.figure(figsize=(7,5))
plt.scatter(df['Temperature'], df['PM2.5'],
            c=df['PM2.5'], cmap='Reds', s=60)
plt.colorbar(label='PM2.5')
plt.title('Sequential Palette')
plt.xlabel('Temperature')
plt.ylabel('PM2.5')
plt.show()
plt.figure(figsize=(7,5))
```

```
plt.scatter(df['Temperature'], df['PM2.5'],
        c=df['PM2.5'], cmap='coolwarm', s=60)
plt.colorbar(label='PM2.5')
plt.title('Diverging Palette')
plt.xlabel('Temperature')
plt.ylabel('PM2.5')
plt.show()
plt.figure(figsize=(7,5))
sns.scatterplot(data=df,
            x='Temperature',
            y='PM2.5',
            hue='Pollution',
            palette='Set2',
            s=80)
plt.title('Qualitative Palette')
plt.show()
print("Comparison of Color Palettes")
print("--------------------------------")
print("Sequential Palette : Best for continuous pollution values and identifying hotspots.")
print("Diverging Palette  : Best for comparing values above and below a reference level.")
print("Qualitative Palette: Best for distinguishing pollution categories.")
print("\nMost Effective Visualization: Sequential Palette because PM2.5 is a continuous variable and higher pollution hotspots are easily identified.")
```

PYTHON OUTPUT :



Comparison of Color Palettes

Sequential Palette : Best for continuous pollution values and identifying hotspots.

Diverging Palette : Best for comparing values above and below a reference level.

Qualitative Palette: Best for distinguishing pollution categories.

Most Effective Visualization: Sequential Palette because PM2.5 is a continuous variable and higher pollution hotspots are easily identified.

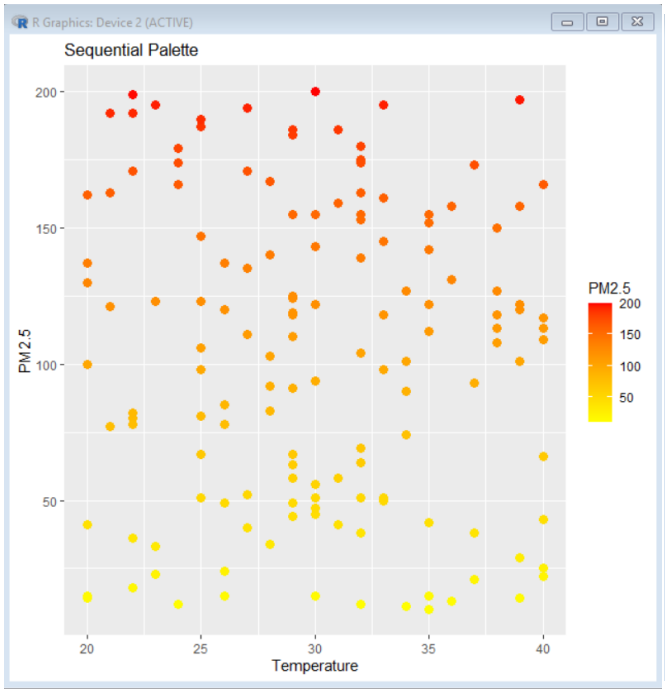
R CODE :

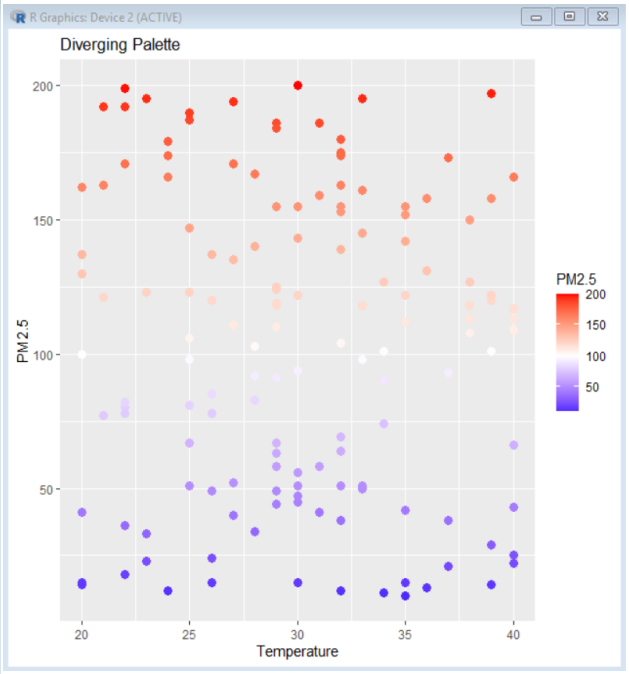
```
library(ggplot2)

library(RColorBrewer)

set.seed(42)
```

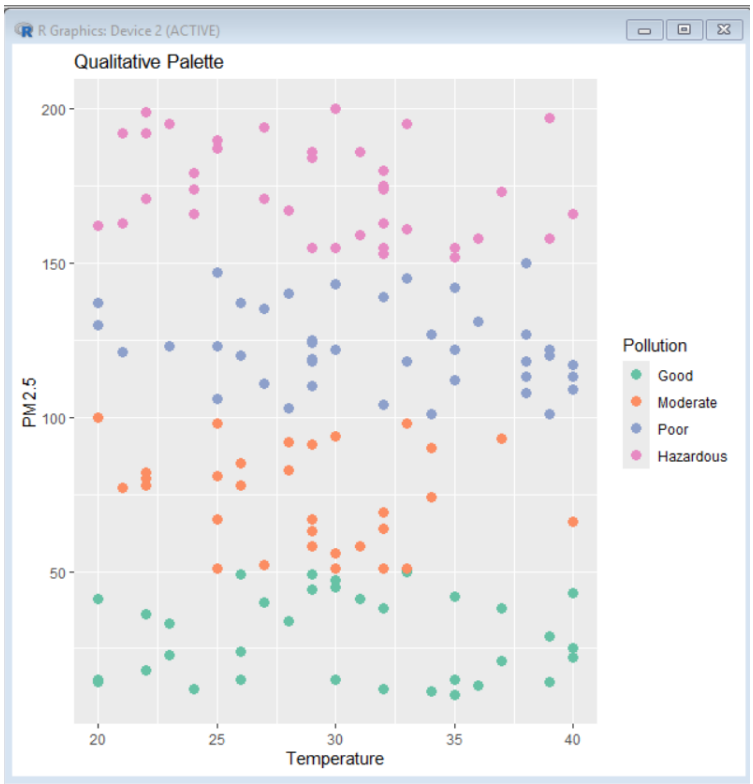
```
df <- data.frame(
  PM2.5 = sample(10:200, 150, replace = TRUE),
  NO2 = sample(5:120, 150, replace = TRUE),
  CO = runif(150, 0.5, 5),
  Temperature = sample(20:40, 150, replace = TRUE),
  Humidity = sample(30:90, 150, replace = TRUE)
)

# Create Pollution Categories
df$Pollution <- cut(
  df$PM2.5,
  breaks = c(0, 50, 100, 150, 250),
  labels = c("Good", "Moderate", "Poor", "Hazardous")
)
ggplot(df, aes(x = Temperature, y = PM2.5, color = PM2.5)) +
  geom_point(size = 3) +
  scale_color_gradient(low = "yellow", high = "red") +
  labs(title = "Sequential Palette",
       x = "Temperature",
       y = "PM2.5")

ggplot(df, aes(x = Temperature, y = PM2.5, color = PM2.5)) +
  geom_point(size = 3) +
  scale_color_gradient2(
    low = "blue",
    mid = "white",
    high = "red",
    midpoint = 100
```

```
  ) +
  labs(title = "Diverging Palette",
       x = "Temperature",
       y = "PM2.5")
ggplot(df, aes(x = Temperature, y = PM2.5, color = Pollution)) +
  geom_point(size = 3) +
  scale_color_brewer(palette = "Set2") +
  labs(title = "Qualitative Palette",
       x = "Temperature",
       y = "PM2.5")
cat("Comparison of Color Palettes\n")
cat("-----------------------------\n")
cat("Sequential Palette : Best for showing continuous pollution values and identifying hotspots.\n")
cat("Diverging Palette  : Best for comparing values above and below a reference point.\n")
cat("Qualitative Palette: Best for distinguishing pollution categories.\n\n")
cat("Most Effective Visualization: Sequential Palette because PM2.5 is a continuous numerical variable and pollution hotspots are clearly highlighted.\n")
```

Output :







SCENARIO 2 : EARTHQUAKE RISK ANALYSIS

PYTHON CODE : import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

```
np.random.seed(42)

n = 1000

df = pd.DataFrame({
    "Magnitude": np.round(np.random.exponential(scale=1.2, size=n) + 0.5, 1),
    "Depth": np.random.randint(1, 701, n),
    "Latitude": np.random.uniform(-90, 90, n),
    "Longitude": np.random.uniform(-180, 180, n),
    "Aftershocks": np.random.poisson(5, n)
})

df["Magnitude"] = df["Magnitude"].clip(0.5, 9.5)

plt.figure(figsize=(8,6))
plt.scatter(df["Magnitude"], df["Depth"], c=df["Aftershocks"], cmap="viridis", s=30)
plt.colorbar(label="Aftershock Frequency")
plt.xlabel("Earthquake Magnitude")
plt.ylabel("Depth (km)")
plt.title("Earthquake Distribution (Linear Scale)")
plt.show()

plt.figure(figsize=(8,6))
plt.scatter(df["Magnitude"], df["Depth"], c=df["Aftershocks"], cmap="viridis", s=30)
plt.yscale("log")
plt.colorbar(label="Aftershock Frequency")
plt.xlabel("Earthquake Magnitude")
```

plt.ylabel("Depth (km) - Log Scale")

plt.title("Earthquake Distribution (Logarithmic Scale)")

plt.show()

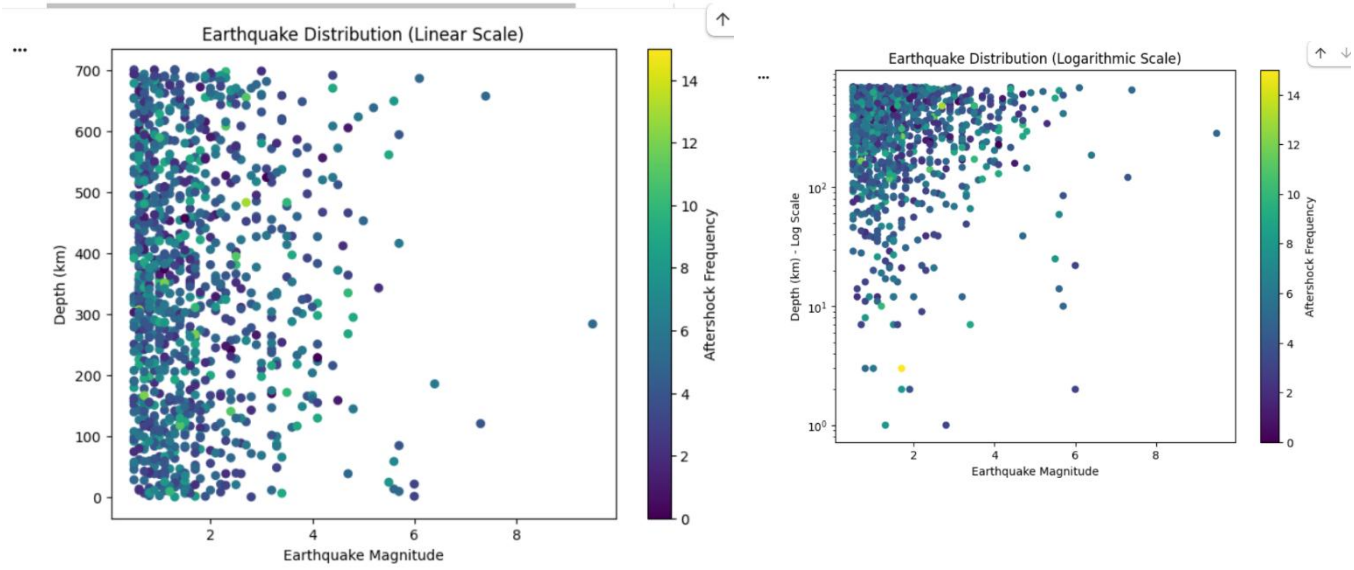
print("Comparison")

print("Linear Scale: Small earthquakes dominate the visualization, making large events harder to distinguish.")

print("Logarithmic Scale: Compresses large depth values and spreads smaller values, improving visibility of earthquake distribution across different depths.")

print("Logarithmic scaling provides a clearer perception when data spans a wide numerical range.")

OUTPUT :



Comparison

Linear Scale: Small earthquakes dominate the visualization, making large events harder to distinguish.

Logarithmic Scale: Compresses large depth values and spreads smaller values, improving visibility of earthquake distribution across different depths.

Logarithmic scaling provides a clearer perception when data spans a wide numerical range.

R CODE : library(ggplot2)

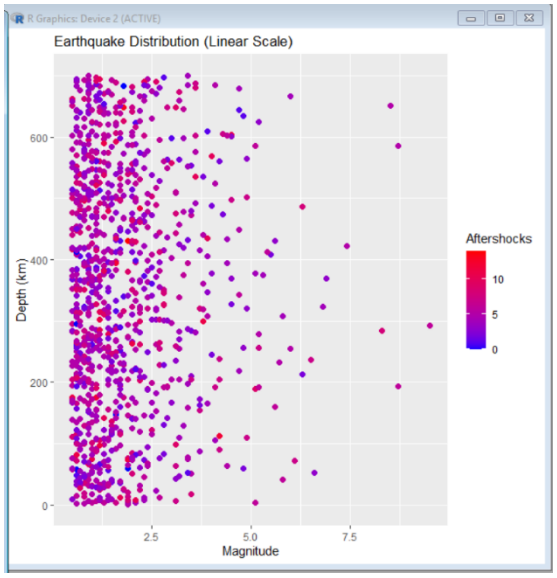
set.seed(42)

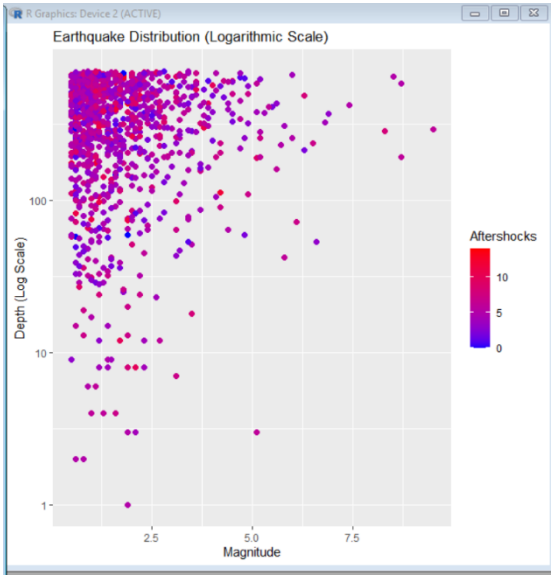
```
n <- 1000

df <- data.frame(
  Magnitude = pmin(round(rexp(n, rate = 1/1.2) + 0.5, 1), 9.5),
  Depth = sample(1:700, n, replace = TRUE),
  Latitude = runif(n, -90, 90),
  Longitude = runif(n, -180, 180),
  Aftershocks = rpois(n, lambda = 5)
)
ggplot(df, aes(x = Magnitude, y = Depth, color = Aftershocks)) +
  geom_point(size = 2) +
  scale_color_gradient(low = "blue", high = "red") +
  labs(
    title = "Earthquake Distribution (Linear Scale)",
    x = "Magnitude",
    y = "Depth (km)"
  )
ggplot(df, aes(x = Magnitude, y = Depth, color = Aftershocks)) +
  geom_point(size = 2) +
  scale_y_log10() +
  scale_color_gradient(low = "blue", high = "red") +
  labs(
    title = "Earthquake Distribution (Logarithmic Scale)",
    x = "Magnitude",
    y = "Depth (Log Scale)"
  )
cat("Comparison\n")
```

cat("Linear Scale: Small earthquakes dominate the visualization, making larger events less distinguishable.\n")

cat("Logarithmic Scale: Compresses larger depth values and spreads smaller values, making the earthquake distribution easier to interpret.\n")

cat("Logarithmic scaling is more effective for datasets with values spanning a wide range.\n")





SCENARIO 3 : WIND DIRECTION ANALYSIS

PYTHON CODE : import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

np.random.seed(42)

n = 365

df = pd.DataFrame({

"WindSpeed": np.random.uniform(2, 30, n),

"WindDirection": np.random.uniform(0, 360, n)

})

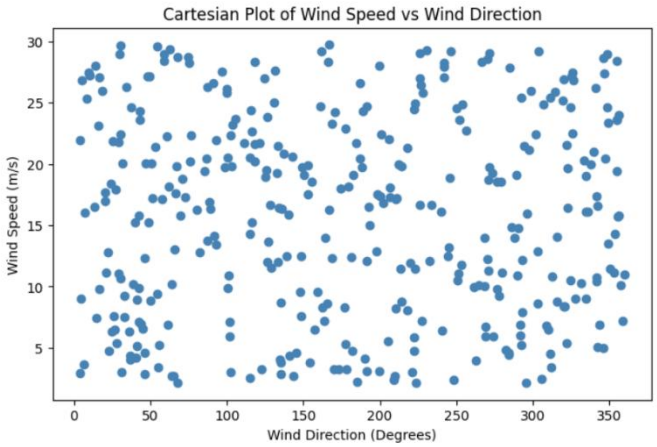
plt.figure(figsize=(8,5))

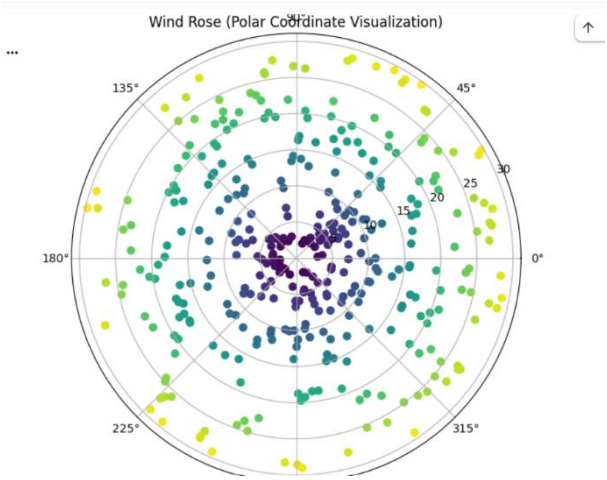
plt.scatter(df["WindDirection"], df["WindSpeed"], color="steelblue")

plt.xlabel("Wind Direction (Degrees)")

```
plt.ylabel("Wind Speed (m/s)")

plt.title("Cartesian Plot of Wind Speed vs Wind Direction")

plt.show()


theta = np.deg2rad(df["WindDirection"])


plt.figure(figsize=(7,7))

ax = plt.subplot(111, polar=True)

ax.scatter(theta, df["WindSpeed"], c=df["WindSpeed"], cmap="viridis", s=40)

ax.set_title("Wind Rose (Polar Coordinate Visualization)")

plt.show()


print("Comparison")

print("Cartesian Plot: Shows the relationship between wind direction and wind speed but makes dominant directions difficult to identify.")

print("Wind Rose: Clearly reveals prevailing wind directions and wind speed patterns.")

print("Recommendation: Wind Rose is more suitable for airport operational decision-making.")
```

OUTPUT :





R CODE : library(ggplot2)

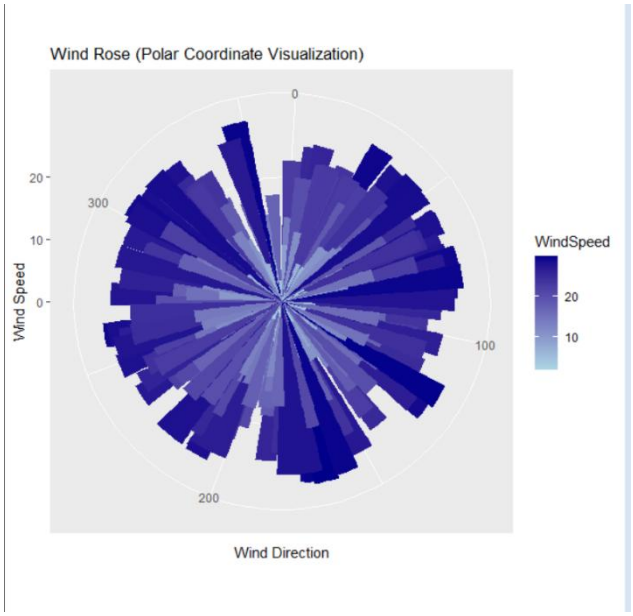
```
set.seed(42)
n <- 365
df <- data.frame(
  WindSpeed = runif(n, 2, 30),
  WindDirection = runif(n, 0, 360)
)

ggplot(df, aes(x = WindDirection, y = WindSpeed)) +
  geom_point(color = "steelblue") +
  labs(
    title = "Cartesian Plot of Wind Speed vs Wind Direction",
    x = "Wind Direction (Degrees)",
    y = "Wind Speed (m/s)"
  )

ggplot(df, aes(x = WindDirection, y = WindSpeed, fill = WindSpeed)) +
  geom_col(width = 8) +
  coord_polar(start = 0) +
  scale_fill_gradient(low = "lightblue", high = "darkblue") +
  labs(
    title = "Wind Rose (Polar Coordinate Visualization)",
    x = "Wind Direction",
    y = "Wind Speed"
  )

cat("Comparison\n")
```

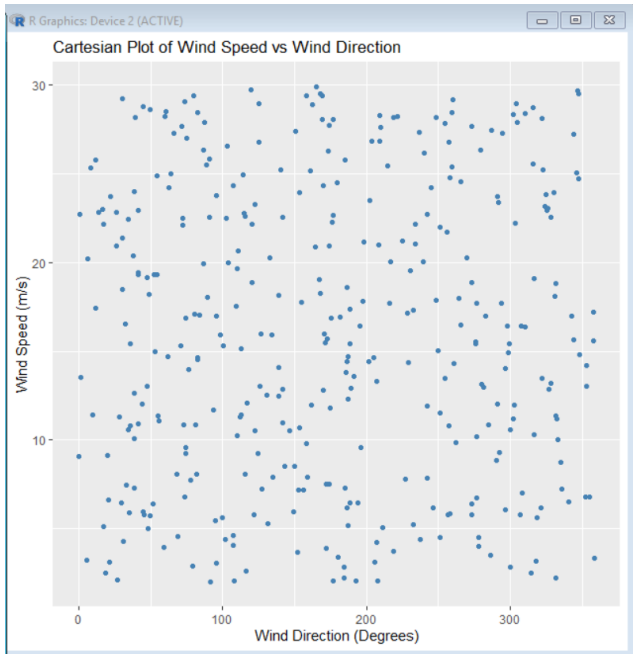
cat("Cartesian Plot: Displays wind speed against direction but does not clearly show prevailing wind directions.\n")

cat("Wind Rose: Clearly identifies dominant wind directions and seasonal wind patterns.\n")

cat("Recommendation: Wind Rose is more effective for airport operational decision-making.\n")

OUTPUT :





SCENARIO 5 : Global Climate Change Study

CODE : import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

np.random.seed(42)

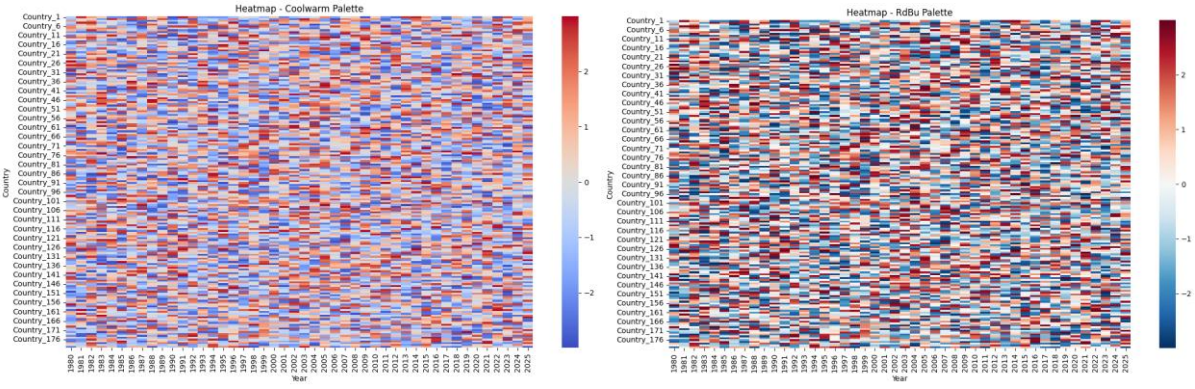
countries = [f"Country_{i}" for i in range(1, 181)]

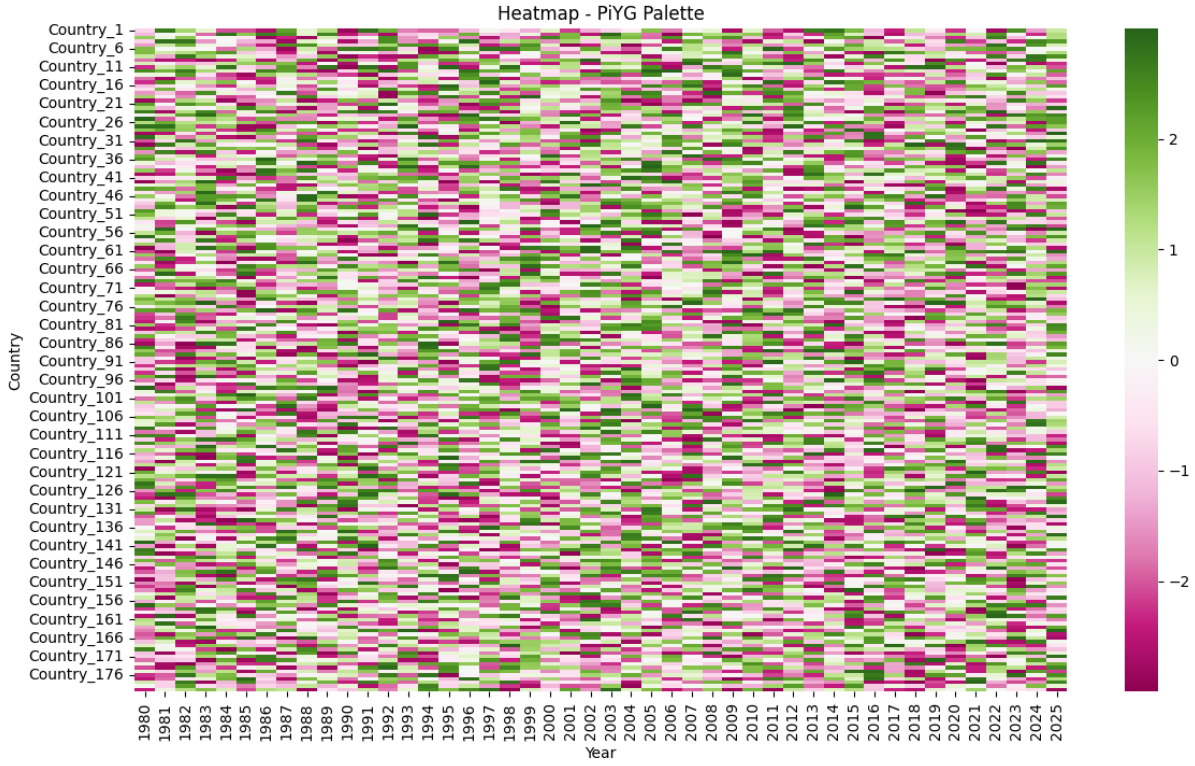
years = list(range(1980, 2026))

data = np.random.uniform(-3, 3, (180, len(years)))

df = pd.DataFrame(data, index=countries, columns=years)

```
plt.figure(figsize=(14,8))
sns.heatmap(df, cmap="coolwarm", center=0)
plt.title("Heatmap - Coolwarm Palette")
plt.xlabel("Year")
plt.ylabel("Country")
plt.show()

plt.figure(figsize=(14,8))
sns.heatmap(df, cmap="RdBu_r", center=0)
plt.title("Heatmap - RdBu Palette")
plt.xlabel("Year")
plt.ylabel("Country")
plt.show()

plt.figure(figsize=(14,8))
sns.heatmap(df, cmap="PiYG", center=0)
plt.title("Heatmap - PiYG Palette")
plt.xlabel("Year")
plt.ylabel("Country")
plt.show()
print("Comparison of Diverging Color Palettes")
print("-------------------------------------")
print("1. Coolwarm: Smooth transition between cool and warm anomalies.")
print("2. RdBu: Clearly distinguishes positive and negative temperature anomalies.")
print("3. PiYG: High contrast but less intuitive for climate data.")
print("\nMost Effective Palette: RdBu because blue naturally represents cooler-than-average temperatures and red represents warmer-than-average temperatures. It is also widely used in climate studies and provides good accessibility.")
```





Comparison of Diverging Color Palettes

1. Coolwarm: Smooth transition between cool and warm anomalies.

2. RdBu: Clearly distinguishes positive and negative temperature anomalies.

3. PiYG: High contrast but less intuitive for climate data.

Most Effective Palette: RdBu because blue naturally represents cooler-than-average temperatures and red represents warmer-than-average temperatures. It is also widely used in climate studies and provides good accessibility.

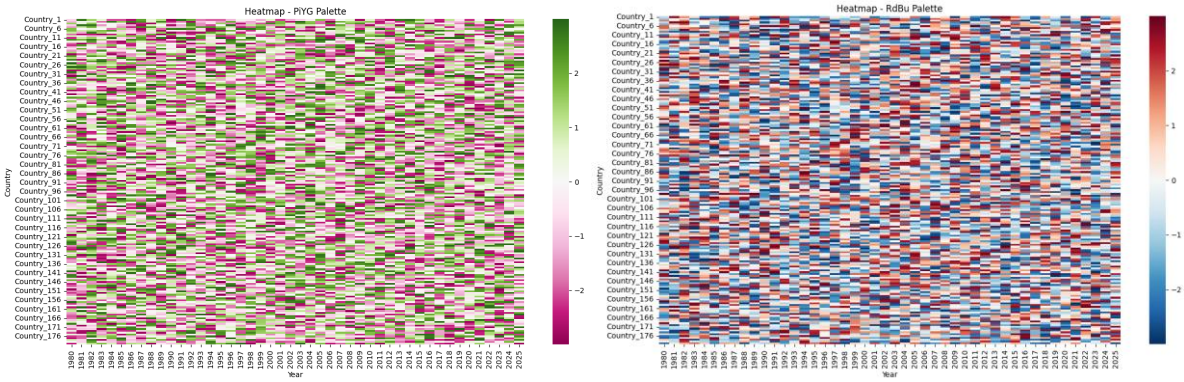
R CODE :

library(ggplot2)

```
library(reshape2)
library(RColorBrewer)

set.seed(42)

countries <- paste0("Country_", 1:180)
years <- 1980:2025

data <- matrix(runif(180 * length(years), -3, 3),
               nrow = 180,
               ncol = length(years))

df <- data.frame(Country = countries, data)
colnames(df)[-1] <- years

climate <- melt(df, id.vars = "Country")
colnames(climate) <- c("Country", "Year", "Anomaly")

ggplot(climate, aes(x = Year, y = Country, fill = Anomaly)) +
  geom_tile() +
  scale_fill_gradient2(low = "blue", mid = "white", high = "red", midpoint = 0) +
  labs(title = "Heatmap - Blue White Red")

ggplot(climate, aes(x = Year, y = Country, fill = Anomaly)) +
  geom_tile() +
  scale_fill_distiller(palette = "RdBu", direction = -1) +
  labs(title = "Heatmap - RdBu")
```

```
ggplot(climate, aes(x = Year, y = Country, fill = Anomaly)) +
  geom_tile() +
  scale_fill_distiller(palette = "PiYG") +
  labs(title = "Heatmap - PiYG")

cat("Comparison of Diverging Color Palettes\n")
cat("---------------------------------------\n")
cat("1. Blue-White-Red: Clearly shows negative and positive anomalies.\n")
cat("2. RdBu: Best for climate anomaly visualization and accessibility.\n")
cat("3. PiYG: Good contrast but less intuitive than RdBu.\n\n")
cat("Most Effective Palette: RdBu because it clearly distinguishes warming and cooling trends while remaining easy to interpret.\n")
```

OUTPUT :



Scenario 6: E-Commerce Sales Analytics (Python)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)

categories = [f"Category_{i}" for i in range(1, 51)]
```

```
regions = [f"Region_{i}" for i in range(1, 31)]

df = pd.DataFrame({
    "Category": np.random.choice(categories, 500),
    "Region": np.random.choice(regions, 500),
    "Sales": np.random.randint(1000, 50000, 500)
})

plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df,
    x="Region",
    y="Sales",
    hue="Category",
    palette="tab20",
    size="Sales",
    sizes=(30,300)
)
plt.xticks(rotation=90)
plt.title("E-Commerce Sales by Category and Region")
plt.show()

pivot = df.pivot_table(values="Sales", index="Category", columns="Region", aggfunc="sum", fill_value=0)

plt.figure(figsize=(12,10))
sns.heatmap(pivot, cmap="YlOrRd")
plt.title("Sales Volume Heatmap")
```

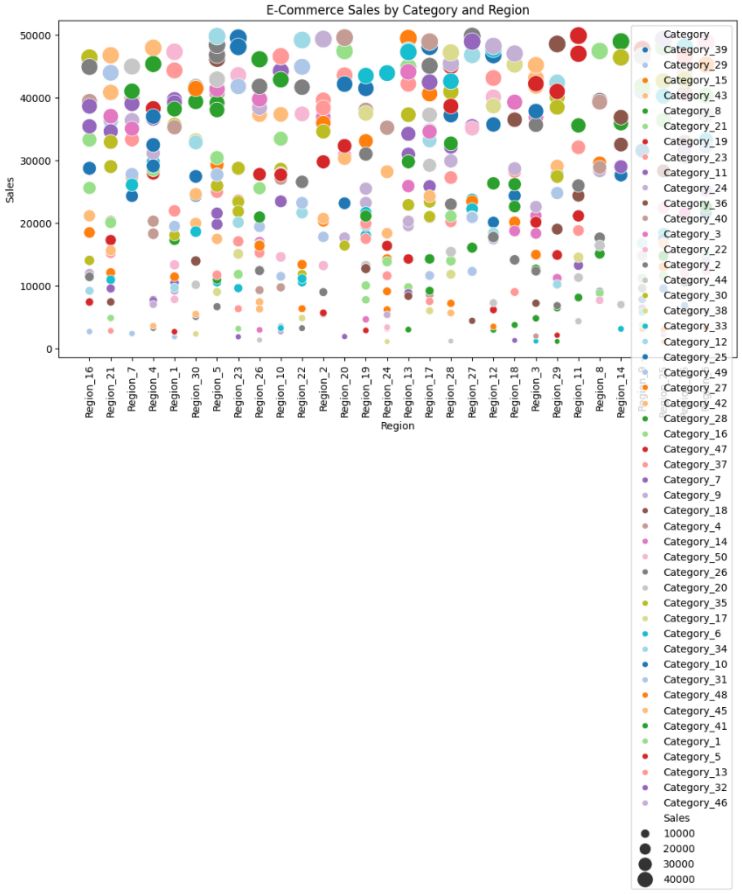
plt.show()

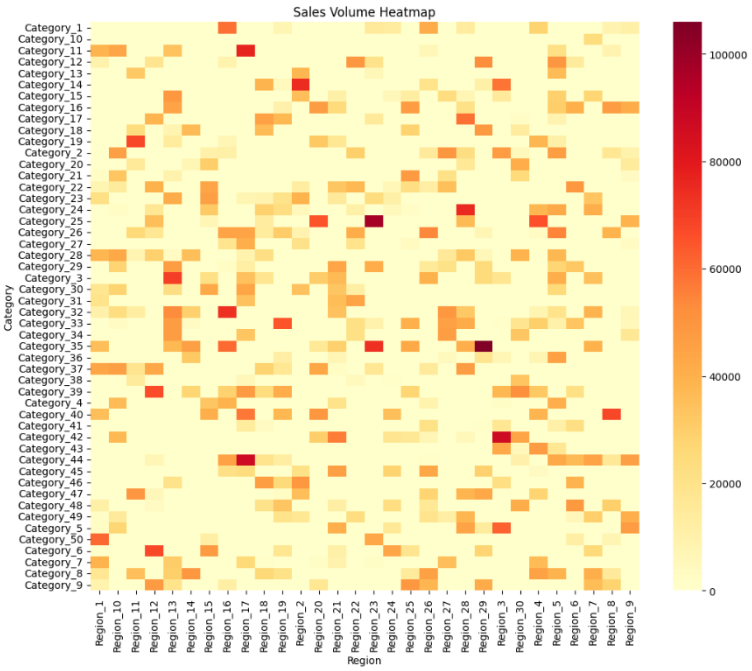
print("Challenges")

print("1. Too many category colors make the visualization cluttered.")

print("2. Using color for both category and sales can confuse users.")

print("3. Size is better used for representing sales volume while color distinguishes categories.")





R CODE: library(ggplot2)

library(tidyr)

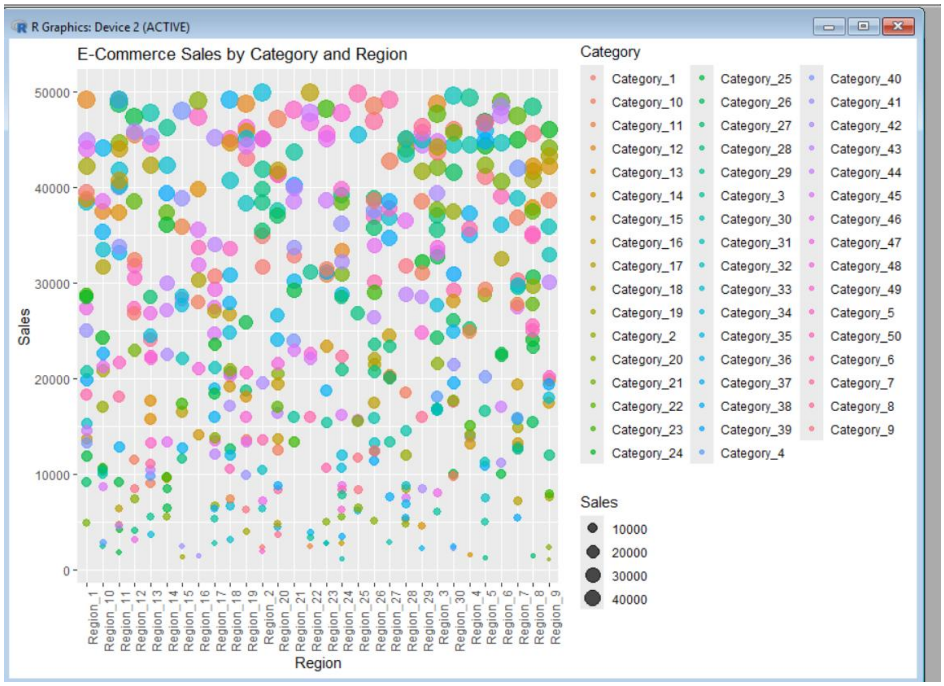
library(dplyr)

set.seed(42)

categories <- paste0("Category_", 1:50)

regions <- paste0("Region_", 1:30)

```
df <- data.frame(
  Category = sample(categories, 500, replace = TRUE),
  Region = sample(regions, 500, replace = TRUE),
  Sales = sample(1000:50000, 500, replace = TRUE)
)

ggplot(df, aes(x = Region, y = Sales, color = Category, size = Sales)) +
  geom_point(alpha = 0.7) +
  labs(title = "E-Commerce Sales by Category and Region") +
  theme(axis.text.x = element_text(angle = 90, hjust = 1))

sales_matrix <- df %>%
  group_by(Category, Region) %>%
  summarise(Sales = sum(Sales), .groups = "drop")

ggplot(sales_matrix, aes(x = Region, y = Category, fill = Sales)) +
  geom_tile() +
  scale_fill_gradient(low = "yellow", high = "red") +
  labs(title = "Sales Volume Heatmap") +
  theme(axis.text.x = element_text(angle = 90, hjust = 1))

cat("Challenges\n")
cat("1. Many category colors make the chart difficult to read.\n")
cat("2. Encoding both category and sales using color may confuse users.\n")
cat("3. Using color for categories and size for sales provides better visual clarity.\n")
```



SCEANRIO 7 : Satellite Image Vegetation Analysis (Python)

CODE : import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

np.random.seed(42)

rows, cols = 50, 50

ndvi = np.random.uniform(-1, 1, (rows, cols))

plt.figure(figsize=(6,6))

sns.heatmap(ndvi, cmap="Greens", cbar_kws={"label":"NDVI"})

plt.title("Sequential Palette - Greens")

plt.show()

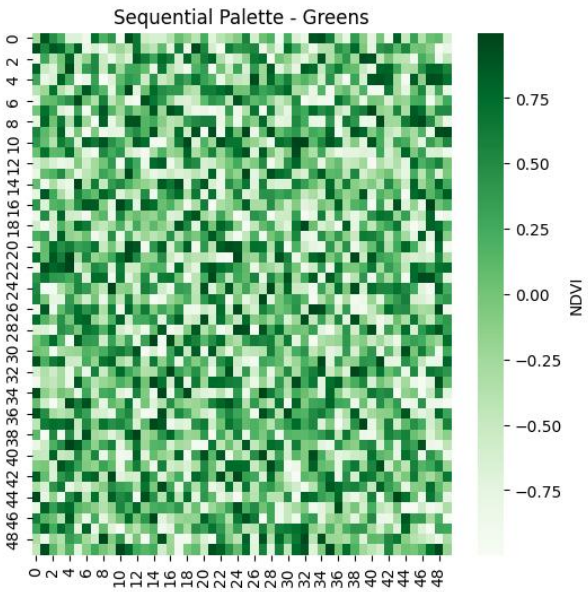
plt.figure(figsize=(6,6))

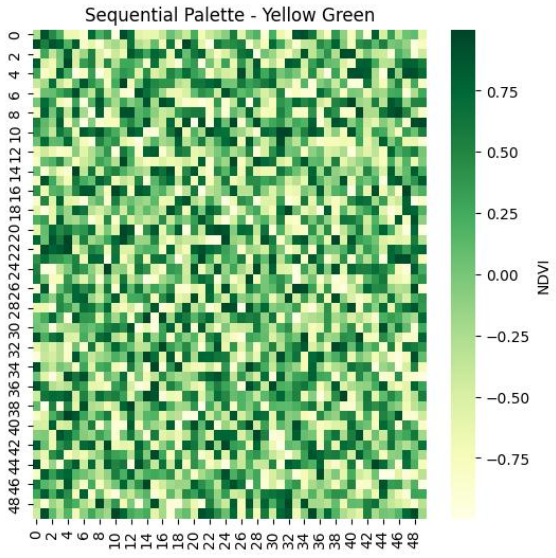
sns.heatmap(ndvi, cmap="YlGn", cbar_kws={"label":"NDVI"})

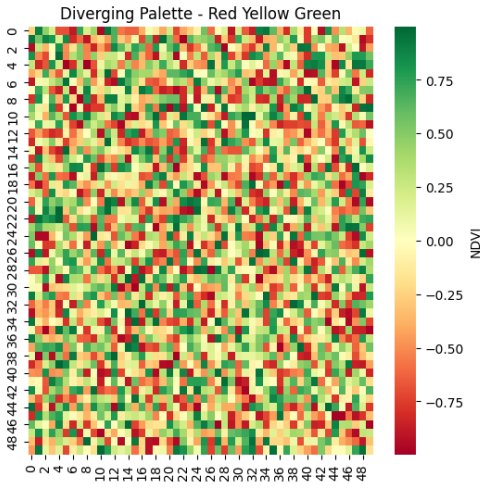
plt.title("Sequential Palette - Yellow Green")

```
plt.show()


plt.figure(figsize=(6,6))
sns.heatmap(ndvi, cmap="RdYlGn", center=0, cbar_kws={"label":"NDVI"})
plt.title("Diverging Palette - Red Yellow Green")
plt.show()


print("Comparison of Color Palettes")
print("----------------------------")
print("Greens: Clearly represents vegetation density but does not distinguish water and barren land well.")
print("Yellow-Green: Suitable for vegetation gradients but less effective for negative NDVI values.")
print("Red-Yellow-Green: Best for NDVI because red indicates barren land, yellow indicates sparse vegetation, and green indicates healthy vegetation.")
print("\nMost Effective Palette: Red-Yellow-Green minimizes interpretation errors and improves readability.")
```

OUTPUT :







Comparison of Color Palettes

Greens: Clearly represents vegetation density but does not distinguish water and barren land well.

Yellow-Green: Suitable for vegetation gradients but less effective for negative NDVI values.

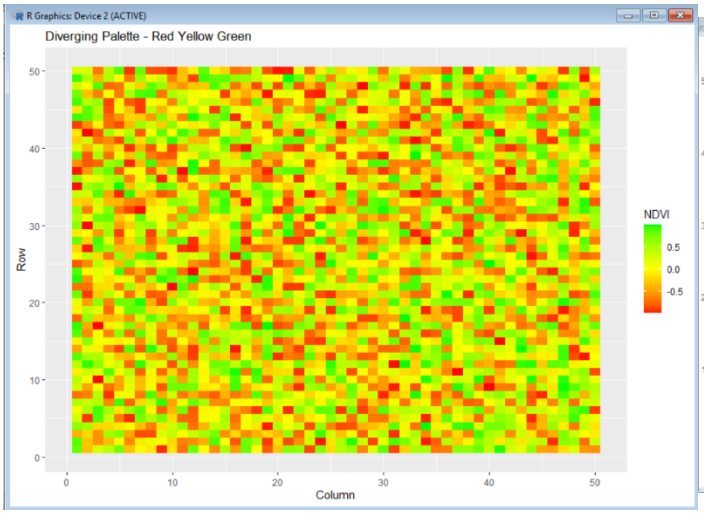
Red-Yellow-Green: Best for NDVI because red indicates barren land, yellow indicates sparse vegetation, and green indicates healthy vegetation.

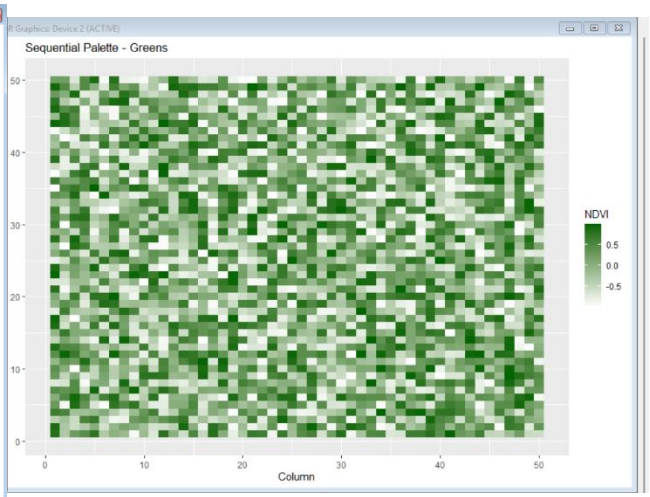
Most Effective Palette: Red-Yellow-Green minimizes interpretation errors and improves readability.

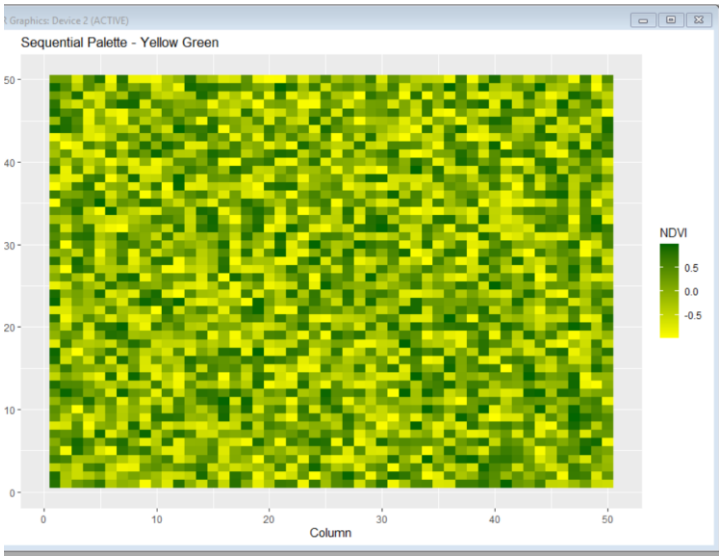
R CODE :

```
library(ggplot2)


set.seed(42)

rows <- 50

cols <- 50

ndvi <- matrix(runif(rows * cols, -1, 1), nrow = rows, ncol = cols)

ndvi_long <- expand.grid(
  Row = 1:rows,
  Column = 1:cols
)


ndvi_long$NDVI <- as.vector(ndvi)
```

```
ggplot(ndvi_long, aes(x = Column, y = Row, fill = NDVI)) +
  geom_tile() +
  scale_fill_gradient(low = "white", high = "darkgreen") +
  labs(title = "Sequential Palette - Greens")

ggplot(ndvi_long, aes(x = Column, y = Row, fill = NDVI)) +
  geom_tile() +
  scale_fill_gradient(low = "yellow", high = "darkgreen") +
  labs(title = "Sequential Palette - Yellow Green")

ggplot(ndvi_long, aes(x = Column, y = Row, fill = NDVI)) +
  geom_tile() +
  scale_fill_gradient2(
    low = "red",
    mid = "yellow",
    high = "green",
    midpoint = 0
  ) +
  labs(title = "Diverging Palette - Red Yellow Green")
cat("Comparison of Color Palettes\n")
cat("----------------------------\n")
cat("Greens: Good for vegetation density but poor at distinguishing negative NDVI values.\n")
cat("Yellow-Green: Suitable for vegetation gradients but less effective for water bodies.\n")
cat("Red-Yellow-Green: Best for NDVI because it clearly separates water/barren land from healthy vegetation.\n")
```







Scenario 8: Cryptocurrency Market Volatility (Python)

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt


np.random.seed(42)


days = pd.date_range("2023-01-01", periods=365)


df = pd.DataFrame({

    "Date": days,
```

```
    "Bitcoin": np.random.uniform(25000, 70000, 365),
    "Ethereum": np.random.uniform(1500, 5000, 365),
    "Solana": np.random.uniform(20, 250, 365),
    "Cardano": np.random.uniform(0.2, 2.5, 365)
})

plt.figure(figsize=(10,6))
plt.plot(df["Date"], df["Bitcoin"], label="Bitcoin", color="orange")
plt.plot(df["Date"], df["Ethereum"], label="Ethereum", color="blue")
plt.plot(df["Date"], df["Solana"], label="Solana", color="green")
plt.plot(df["Date"], df["Cardano"], label="Cardano", color="red")
plt.title("Cryptocurrency Prices (Linear Scale)")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()

plt.figure(figsize=(10,6))
plt.plot(df["Date"], df["Bitcoin"], label="Bitcoin", color="orange")
plt.plot(df["Date"], df["Ethereum"], label="Ethereum", color="blue")
plt.plot(df["Date"], df["Solana"], label="Solana", color="green")
plt.plot(df["Date"], df["Cardano"], label="Cardano", color="red")
plt.yscale("log")
plt.title("Cryptocurrency Prices (Logarithmic Scale)")
plt.xlabel("Date")
plt.ylabel("Log Price")
plt.legend()
plt.show()
```

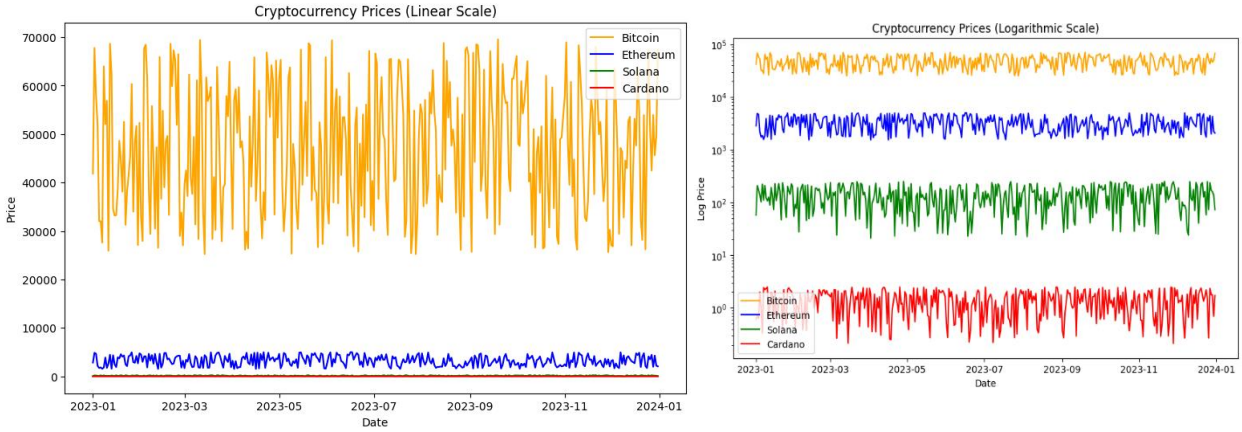
print("Comparison")

print("Linear Scale: Higher-priced cryptocurrencies dominate the chart.")

print("Logarithmic Scale: Makes all cryptocurrencies easier to compare by showing relative changes.")

print("Benefits: Better comparison across different price ranges.")

print("Limitations: Absolute price differences are less obvious.")



Comparison

Linear Scale: Higher-priced cryptocurrencies dominate the chart.

Logarithmic Scale: Makes all cryptocurrencies easier to compare by showing relative changes.

Benefits: Better comparison across different price ranges.

Limitations: Absolute price differences are less obvious.

R CODE : library(ggplot2)

set.seed(42)

days <- seq(as.Date("2023-01-01"), by = "day", length.out = 365)

bitcoin <- runif(365, 25000, 70000)

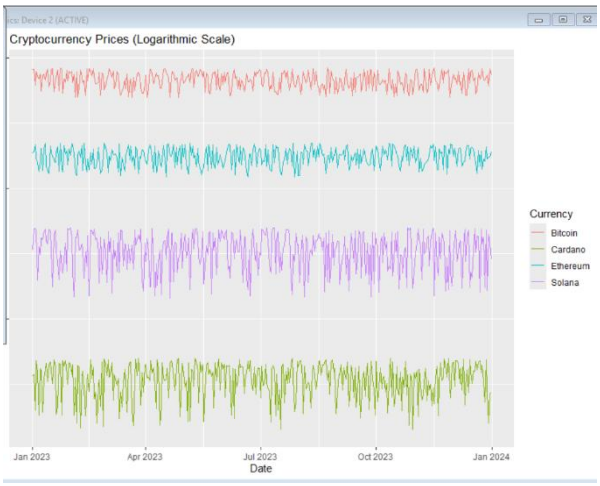
ethereum <- runif(365, 1500, 5000)

```
solana <- runif(365, 20, 250)
cardano <- runif(365, 0.2, 2.5)

crypto <- data.frame(
  Date = rep(days, 4),
  Currency = rep(c("Bitcoin", "Ethereum", "Solana", "Cardano"), each = 365),
  Price = c(bitcoin, ethereum, solana, cardano)
)

ggplot(crypto, aes(x = Date, y = Price, color = Currency)) +
  geom_line() +
  labs(
    title = "Cryptocurrency Prices (Linear Scale)",
    x = "Date",
    y = "Price"
  )

ggplot(crypto, aes(x = Date, y = Price, color = Currency)) +
  geom_line() +
  scale_y_log10() +
  labs(
    title = "Cryptocurrency Prices (Logarithmic Scale)",
    x = "Date",
    y = "Log Price"
  )

cat("Comparison\n")
cat("Linear Scale: Bitcoin dominates because of its much higher price.\n")
```

cat("Logarithmic Scale: Makes all cryptocurrencies easier to compare by showing proportional changes.\n")

cat("Benefits: Useful when values span several orders of magnitude.\n")

cat("Limitations: Absolute price differences become less obvious.\n")



SCENARIO 9: Marathon Runner Performance Analysis (Python)

PYTHON CODE: import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

np.random.seed(42)

n = 200

df = pd.DataFrame({

"Distance": np.linspace(0, 42.2, n),

"Speed": np.random.uniform(8, 18, n),

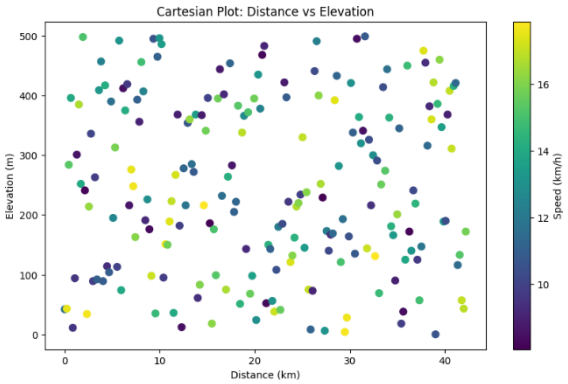
"HeartRate": np.random.randint(110, 190, n),

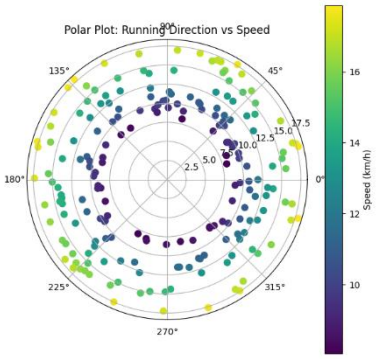
"Elevation": np.random.randint(0, 500, n),

"Direction": np.random.uniform(0, 360, n)

})

```
plt.figure(figsize=(10,6))
plt.scatter(df["Distance"], df["Elevation"],
        c=df["Speed"], cmap="viridis", s=50)
plt.colorbar(label="Speed (km/h)")
plt.xlabel("Distance (km)")
plt.ylabel("Elevation (m)")
plt.title("Cartesian Plot: Distance vs Elevation")
plt.show()

theta = np.deg2rad(df["Direction"])

plt.figure(figsize=(7,7))
ax = plt.subplot(111, polar=True)
sc = ax.scatter(theta, df["Speed"],
            c=df["Speed"], cmap="viridis", s=50)
plt.colorbar(sc, label="Speed (km/h)")
ax.set_title("Polar Plot: Running Direction vs Speed")
plt.show()

print("Comparison")
print("Cartesian Plot: Better for analyzing pacing, distance, and elevation changes.")
print("Polar Plot: Better for visualizing running direction and movement patterns.")
print("Recommendation: Use the Cartesian plot for pacing analysis and the Polar plot for
directional movement insights.")
```





Comparison

Cartesian Plot: Better for analyzing pacing, distance, and elevation changes.

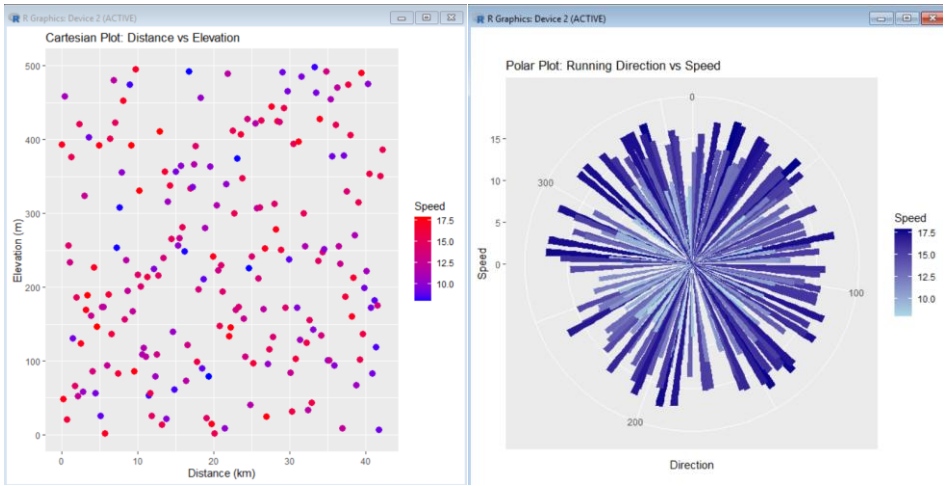
Polar Plot: Better for visualizing running direction and movement patterns.

Recommendation: Use the Cartesian plot for pacing analysis and the Polar plot for directional movement insights.

R CODE : library(ggplot2)

```
set.seed(42)

n <- 200

df <- data.frame(

  Distance = seq(0, 42.2, length.out = n),

  Speed = runif(n, 8, 18),

  HeartRate = sample(110:190, n, replace = TRUE),

  Elevation = sample(0:500, n, replace = TRUE),

  Direction = runif(n, 0, 360)

)


ggplot(df, aes(x = Distance, y = Elevation, color = Speed)) +

  geom_point(size = 3) +

  scale_color_gradient(low = "blue", high = "red") +

  labs(

    title = "Cartesian Plot: Distance vs Elevation",
```

```
    x = "Distance (km)",
    y = "Elevation (m)"
  )
ggplot(df, aes(x = Direction, y = Speed, fill = Speed)) +
  geom_col(width = 3) +
  coord_polar(start = 0) +
  scale_fill_gradient(low = "lightblue", high = "darkblue") +
  labs(
    title = "Polar Plot: Running Direction vs Speed",
    x = "Direction",
    y = "Speed"
  )
cat("Comparison\n")
cat("Cartesian Plot: Best for analyzing pacing, speed, and elevation changes.\n")
cat("Polar Plot: Best for understanding directional movement during the marathon.\n")
cat("Recommendation: Cartesian visualization provides deeper performance insights for coaches, while the Polar plot complements it by showing directional movement.\n")
```

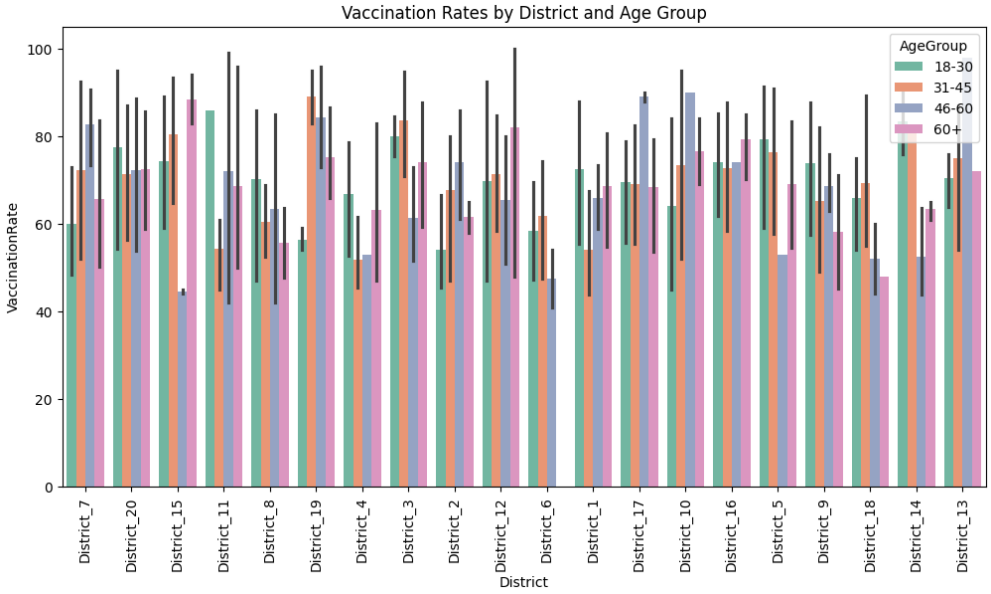


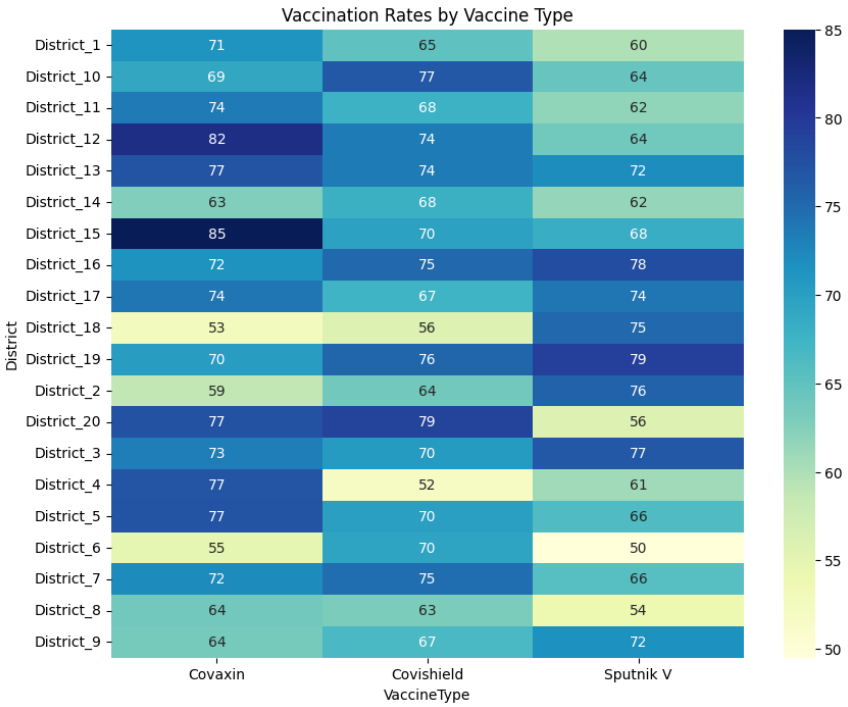
Scenario 10: COVID-19 Vaccination Campaign Dashboard (Python)

```
import numpy as np
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)

districts = [f"District_{i}" for i in range(1, 21)]
age_groups = ["18-30", "31-45", "46-60", "60+"]
vaccine_types = ["Covishield", "Covaxin", "Sputnik V"]

df = pd.DataFrame({
    "District": np.random.choice(districts, 300),
    "AgeGroup": np.random.choice(age_groups, 300),
    "VaccineType": np.random.choice(vaccine_types, 300),
    "VaccinationRate": np.random.randint(40, 101, 300)
})
plt.figure(figsize=(12,6))
sns.barplot(data=df, x="District", y="VaccinationRate", hue="AgeGroup", palette="Set2")
plt.xticks(rotation=90)
plt.title("Vaccination Rates by District and Age Group")
plt.show()
plt.figure(figsize=(10,8))
pivot = df.pivot_table(values="VaccinationRate",
                       index="District",
                       columns="VaccineType",
                       aggfunc="mean")

sns.heatmap(pivot, annot=True, cmap="YlGnBu")
```

```
plt.title("Vaccination Rates by Vaccine Type")
plt.show()

print("Evaluation")
print("Version 1 (Bar Chart): Easy for policymakers to compare age groups across districts.")
print("Version 2 (Heatmap): Quickly identifies high and low vaccination regions.")
print("Accessibility: Use colorblind-friendly palettes such as 'viridis' or 'cividis'.")
print("Highlighting Strategy: Darker shades indicate higher vaccination rates.")
```





R CODE :

```
library(ggplot2)
library(dplyr)

set.seed(42)

districts <- paste0("District_", 1:20)
age_groups <- c("18-30", "31-45", "46-60", "60+")
vaccine_types <- c("Covishield", "Covaxin", "Sputnik V")

df <- data.frame(
  District = sample(districts, 300, replace = TRUE),
  AgeGroup = sample(age_groups, 300, replace = TRUE),
  VaccineType = sample(vaccine_types, 300, replace = TRUE),
  VaccinationRate = sample(40:100, 300, replace = TRUE)
)

ggplot(df, aes(x = District, y = VaccinationRate, fill = AgeGroup)) +
  geom_bar(stat = "summary", fun = "mean", position = "dodge") +
  labs(
    title = "Vaccination Rates by District and Age Group",
    x = "District",
    y = "Vaccination Rate (%)"
  ) +
  theme(axis.text.x = element_text(angle = 90, hjust = 1))

heatmap_data <- df %>%
  group_by(District, VaccineType) %>%
```

```
  summarise(VaccinationRate = mean(VaccinationRate), .groups = "drop")


ggplot(heatmap_data, aes(x = VaccineType, y = District, fill = VaccinationRate)) +
  geom_tile() +
  scale_fill_gradient(low = "lightyellow", high = "darkgreen") +
  labs(
    title = "Vaccination Rates by Vaccine Type",
    x = "Vaccine Type",
    y = "District"
  )


cat("Evaluation\n")
cat("Version 1 (Bar Chart): Suitable for policymakers to compare vaccination rates among age groups.\n")
cat("Version 2 (Heatmap): Helps identify districts with high and low vaccination coverage.\n")
cat("Accessibility: Colorblind-friendly palettes such as viridis or cividis are recommended.\n")
cat("Highlighting Strategy: Darker colors represent higher vaccination rates for quick interpretation.\n")
```